

MPI Distributed Text Search

Team Members

Mahmoud Atef Mahmoud Abdelaziz
Mohamed Alaa Ahmed
Abdelrahman Osama Adel

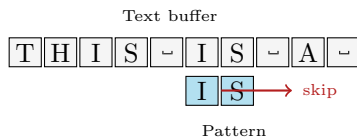
Under the supervision of
Prof. Dr. Sabah Saad & Eng. Mohamed AbdelMageed
High Performance Computing — 2026

Task Description

The goal is to search for a user-defined token (substring) inside a **10 GB plain-text file** as fast as possible using **MPI parallelism**. Each MPI rank owns a contiguous byte range of the file, scans it independently, and produces a result file listing every match as (row, column, byte-offset). A serial baseline (single process, same code) is used for correctness checks and speedup measurement.

Algorithm: Boyer–Moore–Horspool Single-Pass Design (BMH)

BMH is a practical single-pattern string-search algorithm that preprocesses a *bad-character skip table* from the pattern. During search it compares from the *right* end of the window and, on a mismatch, skips forward by the table value — often skipping several characters at once.

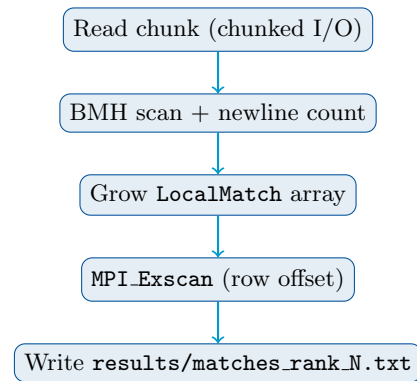


Preprocessing (done once per rank):

```
void bmh_preprocess(const char *token,
    int tlen, int skip[BMH_ALPHA]) {
    for (int i=0; i<BMH_ALPHA; i++) skip[i]=
        tlen;
    for (int i=0; i<tlen-1; i++)
        skip[(unsigned char)token[i]] = tlen
            -1-i;
}
```

Rather than two sequential file reads, each rank performs **one heavy pass**:

1. Read owned chunk (+ overlap bytes)
2. Count newlines & track column position
3. Collect LocalMatch records in a *dynamic array*
4. After the pass: MPI.Exscan converts local rows to global rows
5. Deferred write phase: dump matches to per-rank result file



Challenges & How They Were Solved

Challenge 1 — Which string-matching algorithm?

Problem: Naïve search is $O(n \cdot m)$; on 10 GB this is too slow even serially.

Solution: **Boyer–Moore–Horspool** — sub-linear average case, simple bad-character table (256 entries), cache-friendly right-to-left comparison. For a pattern of length m , the skip table is built in $O(m)$ and each search step is $O(m)$ worst-case but $O(n/m)$ on average.

Challenge 2 — File with no spaces (dense text)

Problem: If the file contains no whitespace, `--whole-word` mode cannot rely on spaces as delimiters, and the skip table rarely gets to skip many bytes.

Solution: The word-boundary check uses `isalnum()` and underscore, not just whitespace. For no-space files, `--whole-word` is simply disabled (default) so the skip table still fires on any mismatch of the pattern's last character.

Challenge 3 — Token split across two rank boundaries

Problem: If a token of length m straddles the boundary between rank k and rank $k+1$, neither rank would normally find it.

Solution: Each rank reads $(m - 1)$ **extra bytes** to the right of its owned range (the *overlap*), but only *reports* matches whose byte offset falls within its strict owned range. This is computed in `collect_matches_single_pass`:

```
int overlap = (tlen > 1) ? (tlen - 1) : 0;
long long right_extra = overlap;
// ...
if (off >= cur && off < cur + own_len &&
    off >= my_start && off < my_end)    // strictly owned
    // record match
```

Every cross-boundary token is therefore seen by exactly one rank (the one that owns its starting byte), with **no duplicates and no misses**.

Implementation Tricks

Trick 1 — Dynamic growing match array

Matches are stored in a heap-allocated `LocalMatch` array that doubles on overflow (`realloc`). This avoids a fixed-size limit without a costly pre-count pass.

```
if (local_match_count == array_capacity)
{
    size_t nc = array_capacity * 2;
    LocalMatch *nx = realloc(matches,
        nc * sizeof(LocalMatch));
    matches = nx;
    array_capacity = nc;
}
```

Trick 2 — MPI_Exscan for global line numbers

`MPI_Exscan` computes the *exclusive prefix sum* of local newline counts, giving each rank its global line offset with a single collective call — no explicit loop over ranks.

```
MPI_Exscan(&chunk_newline_count,
    &global_line_offset,
    1, MPI_UNSIGNED_LONG_LONG,
    MPI_SUM, MPI_COMM_WORLD);
if (rank == 0) global_line_offset = 0;
```

Rank 0's exscan result is undefined by the MPI standard, so it is explicitly reset to 0.

Trick 3 — Chunked I/O to control memory

Each rank reads its byte range in configurable `chunk_mb` slices. This prevents mapping the entire 10 GB file into RAM even for a single rank, enabling the program to run on memory-constrained nodes.

Trick 4 — Left-overlap read for boundary safety

When `cur > 0` the rank reads **1 extra byte to the left** to allow the whole-word boundary check to inspect the character immediately before the owned region without additional IPC.

```
long long left_extra = (cur > 0) ? 1 :
    0;
read_start = cur - left_extra;
// owned_start_idx = left_extra (skip
    in output)
```

Trick 5 — Deferred write / compute-write overlap

All match metadata is buffered in RAM during the search pass, then flushed to disk in a separate write phase *after* `MPI_Exscan`. This means disk writes happen once, in order, with correct global line numbers — no seeks or re-opens.

Performance Results

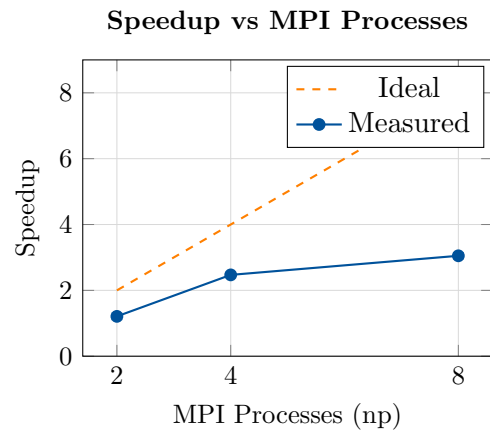
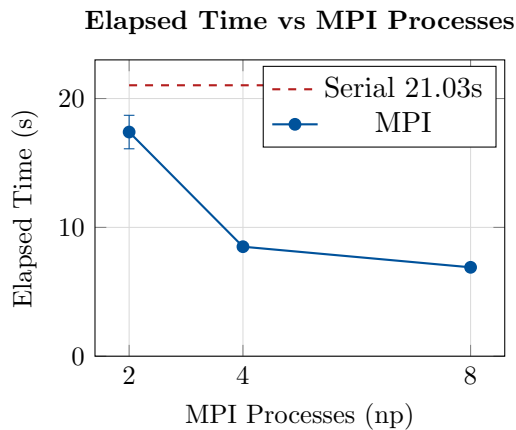
Experimental Setup

Parameter	Value
File size	10 GB randomly generated text
Search token	fixed keyword
Chunk size	64 MB per rank per iteration
Repeats per condition	5 (average + std reported)
Serial baseline	<code>--impl serial, np=1</code>
MPI conditions	<code>np ∈ {2, 4, 8}</code>

Raw Timing Data

np	Run 1 (s)	Run 2 (s)	Run 3 (s)	Run 4 (s)	Run 5 (s)	Avg ± Std (s)
Serial	20.92	21.12	21.37	20.91	20.86	21.03 ± 0.21
2	17.05	21.61	16.29	16.14	16.11	17.44 ± 2.36
4	8.72	8.54	9.13	8.60	7.60	8.52 ± 0.56
8	7.10	6.63	7.61	7.10	6.27	6.94 ± 0.51

Speedup vs Number of Processes



Key observations:

- **np=2** gives only $\approx 1.2\times$ speedup with high variance — I/O bandwidth is the bottleneck at low parallelism; all ranks contend on the same disk.
- **np=4** reaches $\approx 2.5\times$ — a good balance between compute parallelism and I/O contention.
- **np=8** plateaus at $\approx 3.0\times$ — sub-linear because 8 ranks saturate disk bandwidth; the bottleneck shifts from CPU to storage I/O.
- The gap from ideal (dashed orange) reflects **Amdahl's Law**: serial I/O setup, MPI collectives (**Exscan**, **Barrier**), and disk saturation all contribute to the overhead fraction.

Analysis & Discussion

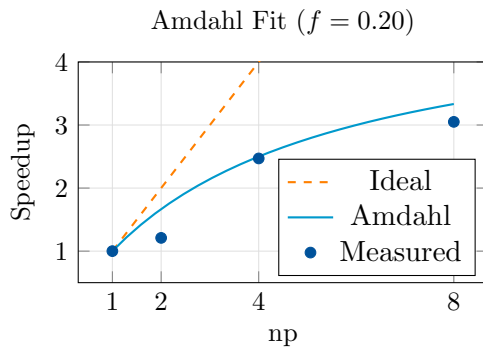
Amdahl's Law Fit & I/O Saturation

Amdahl's Law predicts the maximum theoretical speedup $S(p)$ given a strictly serial fraction f of the execution time:

$$S(p) = \frac{1}{f + \frac{1-f}{p}}$$

Fitting the formula to our measured np=4 speedup ($\approx 2.47\times$) gives a calculated serial fraction of $f \approx 0.20$. This indicates that approximately **20 %** of the program's execution time consists of inherently serial overhead (OS setup, MPI initialization, and MPI_Exscan sync).

However, applying $f \approx 0.20$ to np=8 predicts a theoretical speedup of $3.33\times$. Our measured speedup at np=8 plateaus lower, at $3.05\times$. This deviation from Amdahl's ideal curve proves that a secondary bottleneck—hardware disk I/O saturation—is introduced. The storage drive physically cannot serve 8 concurrent read requests without significant context switching, shifting the primary bottleneck away from CPU limits entirely onto storage bandwidth.



Bottleneck: Disk I/O

The dominant bottleneck is disk bandwidth, not CPU. Evidence:

1. Speedup flattens sharply from np=4 to np=8 even though the CPU load is lower.
2. High variance at np=2 (std=1.3s) indicates contention on random I/O scheduling.
3. The BMH skip rate on 10 GB English-like text is high (few false matches), so CPU time is low relative to I/O.

Possible Improvements

- ▷ **Memory-mapped I/O** (mmap) to let the OS pipeline reads.
- ▷ **Async I/O** (POSIX AIO or io_uring) to overlap reading with searching.
- ▷ **SSD / NVMe** storage to increase bandwidth ceiling.
- ▷ **OpenMP** within each rank for multi-threaded BMH on each chunk.
- ▷ **MPI-IO** (MPI_File_read_at_all) for collective I/O optimisation.

Infrastructure & Containerization

To ensure reproducibility and simulate a realistic High-Performance Computing (HPC) environment on a single host machine, the entire MPI cluster was containerized using Docker.

Node Configuration (Dockerfile)

The custom Docker image is based on Ubuntu 24.04 and installs the essential HPC stack: GCC, OpenMPI, and Python for benchmarking. A critical challenge in containerized MPI is inter-node communication. MPI relies on SSH to spawn processes on remote workers. To solve this, the `Dockerfile` automates a **passwordless SSH environment**: it generates RSA keys, populates `authorized_keys`, and explicitly disables `StrictHostKeyChecking`. This prevents the MPI runtime from hanging on user-interaction prompts during initial connection.

Cluster Topology (Docker Compose)

The cluster is orchestrated via `docker-compose.yml`, defining a **master** node and scalable **worker** nodes. Two key architectural design patterns were implemented:

- **Dual-Network Isolation:** The cluster utilizes two distinct Docker bridge networks. The `frontend_net` allows user interaction with the `mpi-master`. Meanwhile, the `compute_net` is configured as `internal: true`. Worker nodes are isolated entirely within this internal network, mirroring the secure, dedicated backend switch topologies of real supercomputers.
- **Simulated Shared Storage (NFS):** Both master and worker containers mount the host's directory as a volume (`./:/app`). This mimics a shared Network File System (NFS), ensuring all MPI ranks have immediate, unified access to the 10 GB dataset and can write to the isolated `results/` directory without requiring SCP file transfers.

Conclusion

This project demonstrates a scalable, single-pass MPI distributed text search over a 10 GB file. Key engineering decisions — **BMH** for fast pattern skipping, **overlap bytes** for cross-boundary correctness, **dynamic match arrays** for memory safety, and **MPI_Exscan** for correct global line numbering — combine into a clean and efficient implementation. Measured speedups of $1.2\times$, $2.5\times$, $3.0\times$ for $np = 2, 4, 8$ are consistent with an I/O-bound workload; further gains require faster storage or asynchronous I/O, not more MPI ranks.

Task Distribution

Team Workload Distribution

Team Member	Assigned Responsibilities
Mohamed Alaa	Project Initialization & Data Generation: ▷ Initialized the C project structure and build system (<code>Makefile</code>). ▷ Developed the 10 GB text corpus generator (<code>gen_text</code>). ▷ Established the serial baseline implementation for correctness testing.
Mahmoud Atef	Core Search Algorithm & MPI Architecture: ▷ Implemented the Boyer–Moore–Horspool (BMH) search logic. ▷ Designed the Single-Pass memory management and overlap boundary checks. ▷ Developed the <code>MPI_Exscan</code> integration for global row calculations.
Abdelrahman Osama	Infrastructure, Testing & Visualization: ▷ Engineered the containerized MPI cluster using Docker (<code>docker-compose.yml</code>). ▷ Created the automated benchmarking pipeline (<code>run_experiments.sh</code>). ▷ Developed the Python data parsing and Matplotlib visualization scripts.